

Stage 6 – Amazon ECR: Container Registry

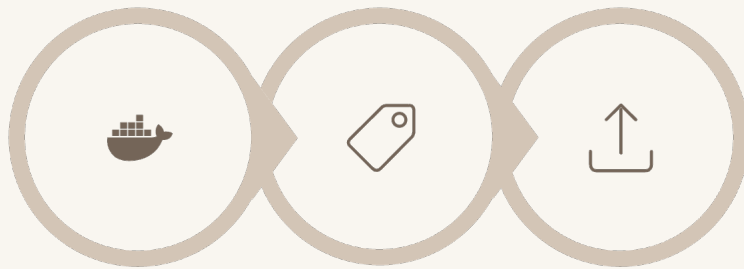
CI/CD PIPELINE SERIES

What is Amazon ECR?

Amazon Elastic Container Registry (ECR) is a fully managed private Docker container registry that stores, manages, and deploys container images securely at scale.

Core Capabilities

- **Private Repository** – Isolated, access-controlled image storage
- **Image Versioning** – Tag-based version tracking per build
- **Image Lifecycle** – Automated cleanup of old/unused images
- **Benefits** – Native AWS IAM integration, high availability, encryption at rest

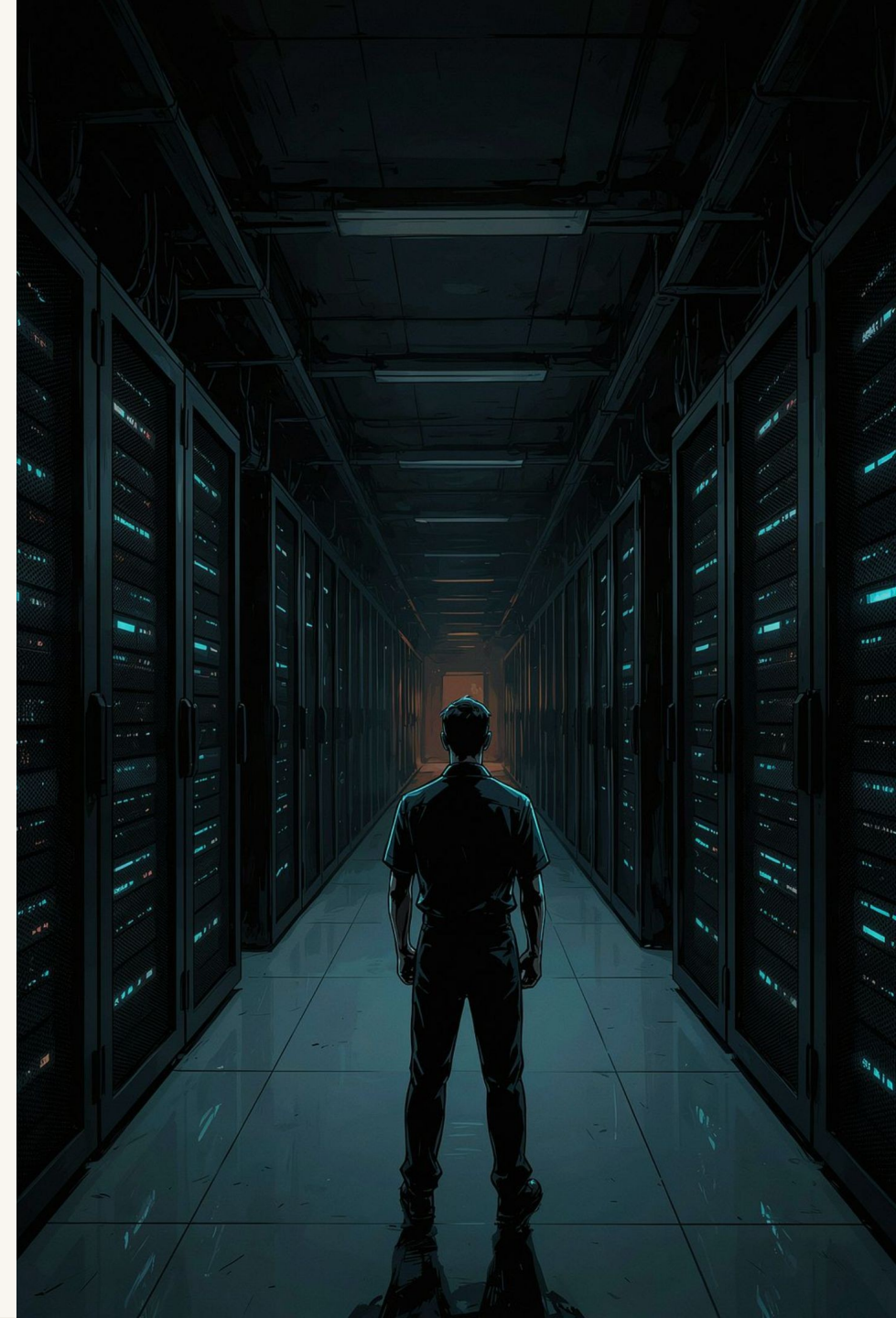


Build Image

Tag with ECR

Push to ECR

Every Jenkins build tags the Docker image with the build number and pushes it to ECR, ensuring a clean, versioned artifact store ready for deployment.



Stage 7 – Docker Compose Deployment

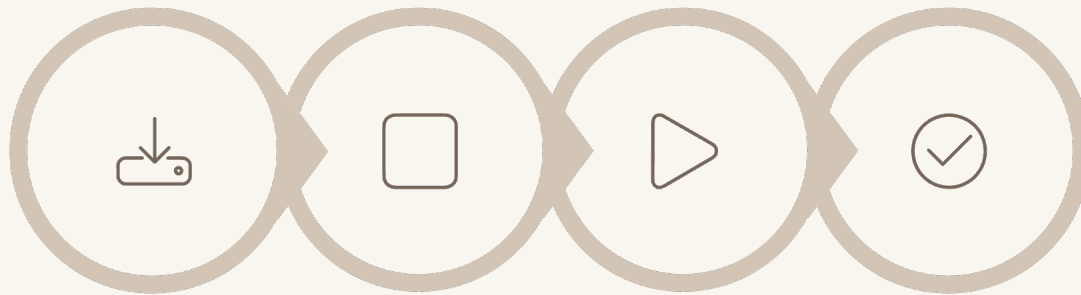
DEPLOYMENT STAGE

Deployment Commands

- **docker compose pull** – Fetches latest images from ECR
- **docker compose up** – Recreates containers with updated images
- **Container Recreation** – Zero-downtime rolling replacement

Configuration Essentials

- **Networking** – Isolated bridge networks per stack
- **Volumes** – Persistent data across container restarts
- **Restart Policies** – `always` or `unless-stopped` for resilience

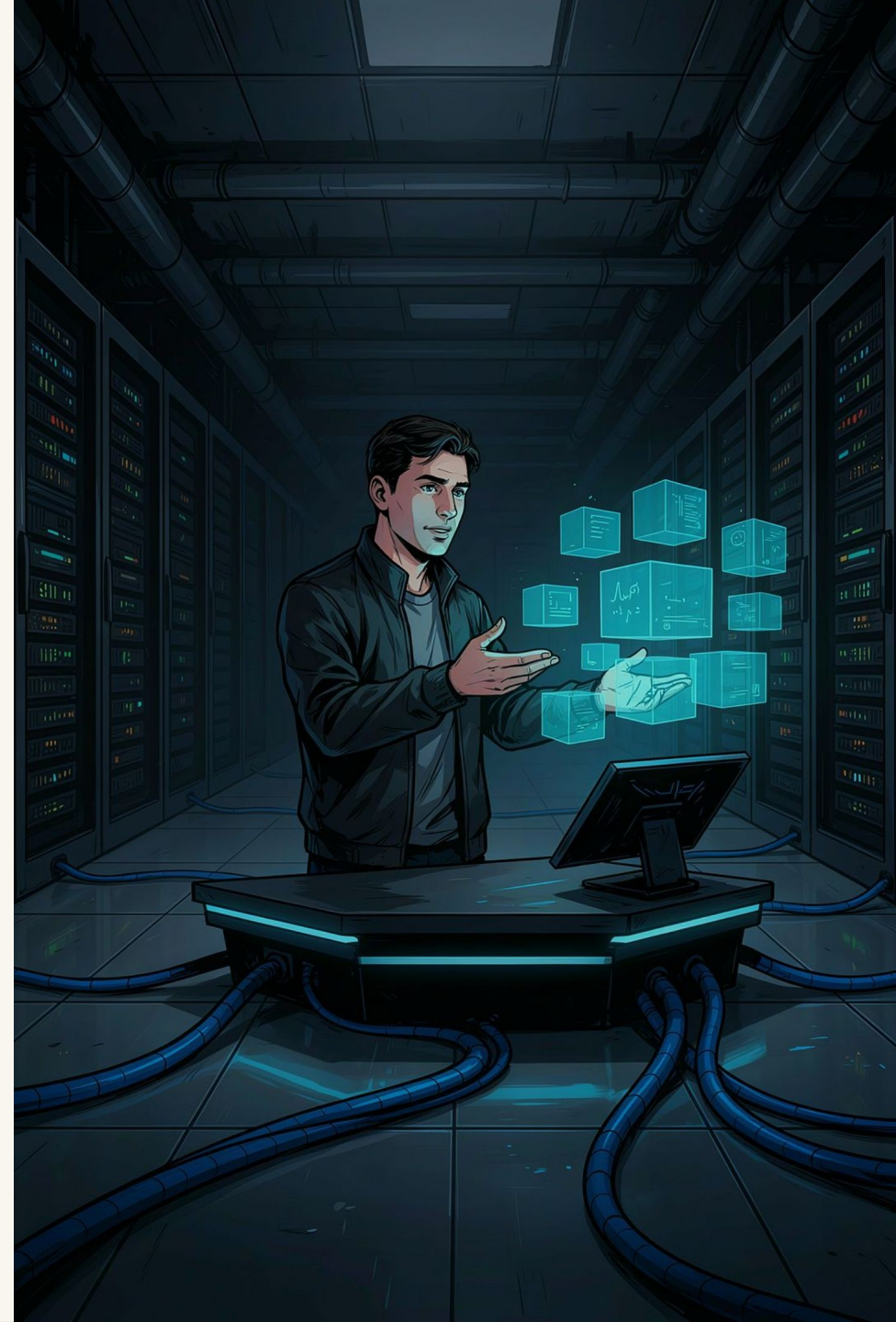


Pull Image

Stop Old

Start New

Verify





Health Validation

POST-DEPLOYMENT

App Health Check

HTTP endpoint or script confirms the application is responding correctly after deployment.

Container Status

Run `docker ps` to verify all containers are in a running state with correct uptime.

Docker Logs

Inspect `docker logs` output for startup errors, exceptions, or connection failures.

Failure & Recovery

On failure detection, Jenkins marks the build failed and triggers rollback or alert procedures.



Run Health
Check

Inspect Logs

Pass or Alert

Email Notifications & Pipeline Visibility

OBSERVABILITY

Notification Triggers

- **Build Success** – Confirms image built and pushed to ECR
- **Build Failure** – Alerts team with stage name and error log
- **Deployment Status** – Reports whether containers started successfully
- **Pipeline Reports** – Full summary of all stages and durations

Sample Email Content

Subject

Jenkins Build #42 –
SUCCESS | my-app:latest
deployed

Body

Stage: Deploy | Duration:
3m 12s | ECR Image:
123456.dkr.ecr.us-east-
1.amazonaws.com/my-
app:42

Jenkins SMTP plugin sends automated emails at pipeline completion, giving full visibility without manual monitoring.

Jenkins Credentials Management

SECURITY



GitHub Token

Personal access token stored as a Jenkins secret for authenticated repository cloning and webhook triggers.



AWS Credentials

Access Key ID and Secret stored as AWS credentials type — used for ECR authentication via `aws ecr get-login-password`.

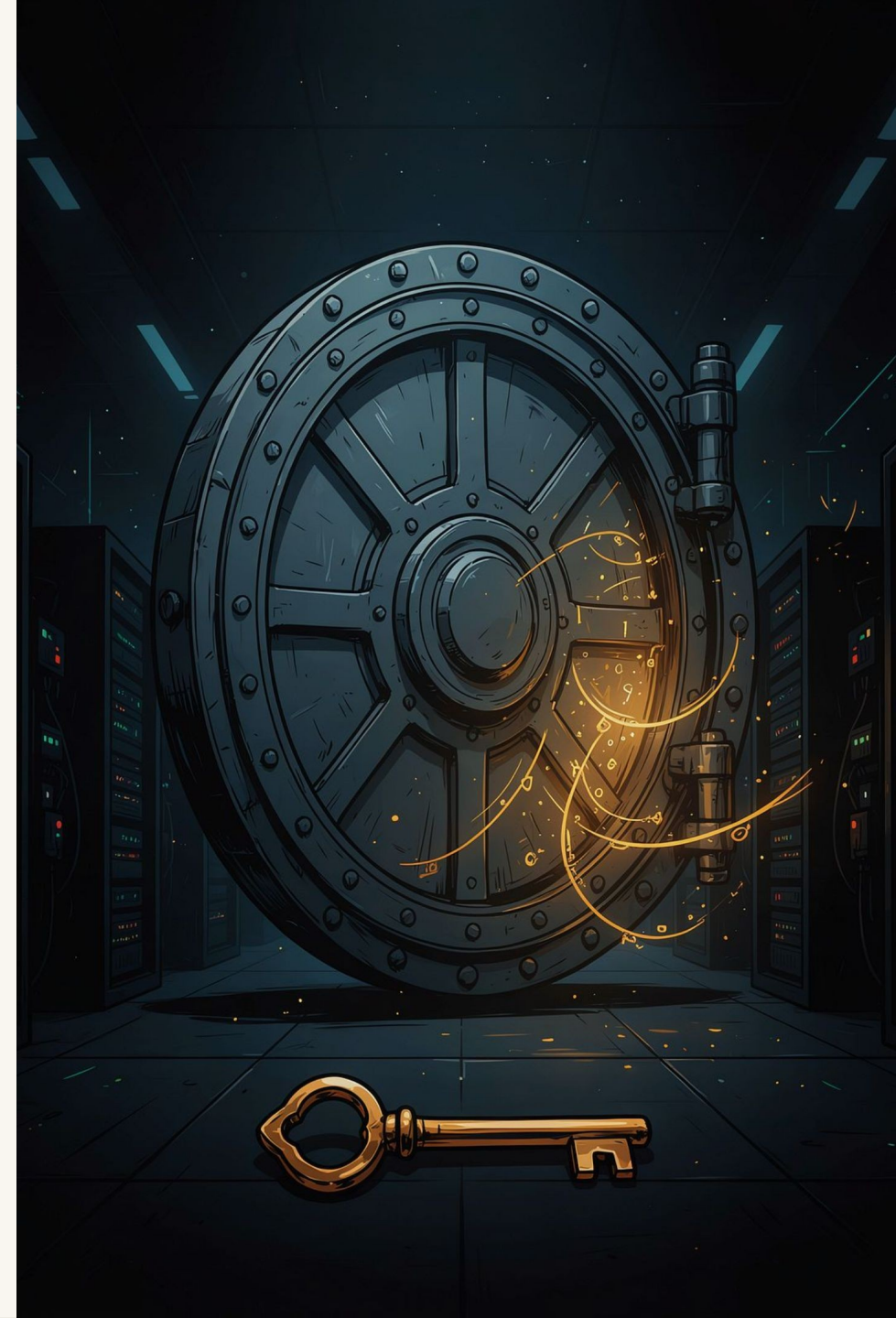


Sonar & SMTP Tokens

SonarQube token for code analysis API and SMTP credentials for email notifications — never hardcoded in Jenkinsfile.



Security Best Practice: Always use `withCredentials([])` blocks in Jenkinsfile. Never expose secrets in console output or environment variables.



Docker Compose Deep Dive

CONFIGURATION

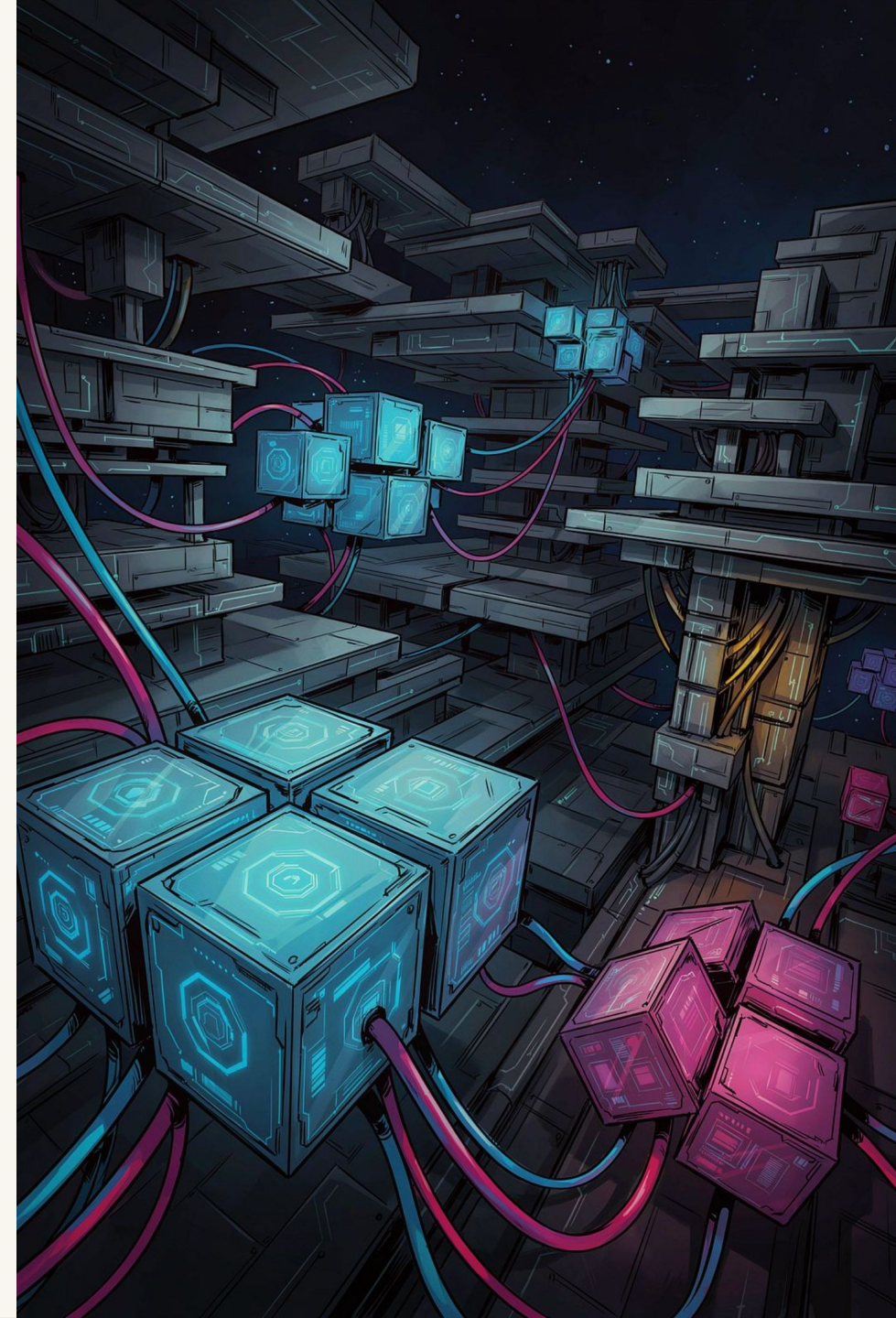
Key Directives

- **Networks** – Custom bridge networks isolate service communication
- **Volumes** – Named volumes persist Redis data and app state
- **Environment Variables** – Injected via `.env` file or shell exports
- **depends_on** – Enforces startup order (e.g., Redis before Python app)

Resilience Settings

- **Restart Policies** – `always` ensures auto-recovery on crash
- **Health Checks** – Built-in `healthcheck` block validates readiness

Docker Compose orchestrates multi-container stacks declaratively, making deployments reproducible and environment-agnostic across dev, staging, and production.



Amazon ECR Deep Dive

REGISTRY ARCHITECTURE

1

Repositories

One repo per application. Namespace with project and environment (e.g., `myapp/prod`).

2

Authentication

Use `aws ecr get-login-password` piped to `docker login`. Token valid for 12 hours.

3

Tagging Strategy

Tag images with Git commit SHA and build number for full traceability. Always maintain a `latest` tag.

4

Image Cleanup

Lifecycle policies auto-expire untagged images older than 30 days, reducing storage costs.

5

Enterprise Best Practices

Enable image scanning on push, enforce immutability on release tags, and use cross-account replication for multi-region deployments.

Production Best Practices

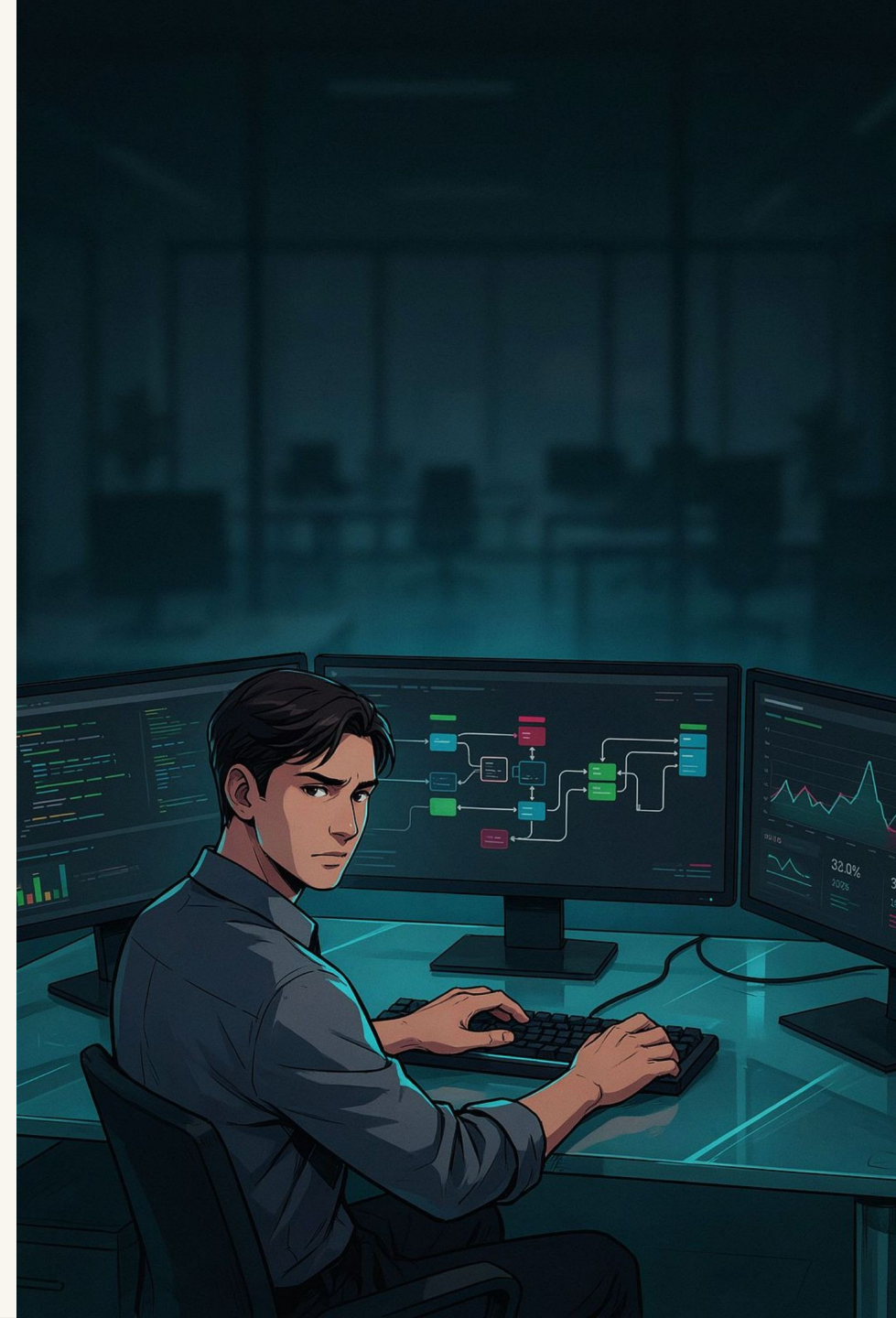
ENTERPRISE STANDARDS

Pipeline Design

- **Parameterized Pipelines** – Accept branch, environment, and version inputs
- **Pipeline as Code** – Jenkinsfile committed to source control
- **Immutable Images** – Never overwrite a released tag in ECR
- **Versioning** – Semantic versioning tied to Git tags

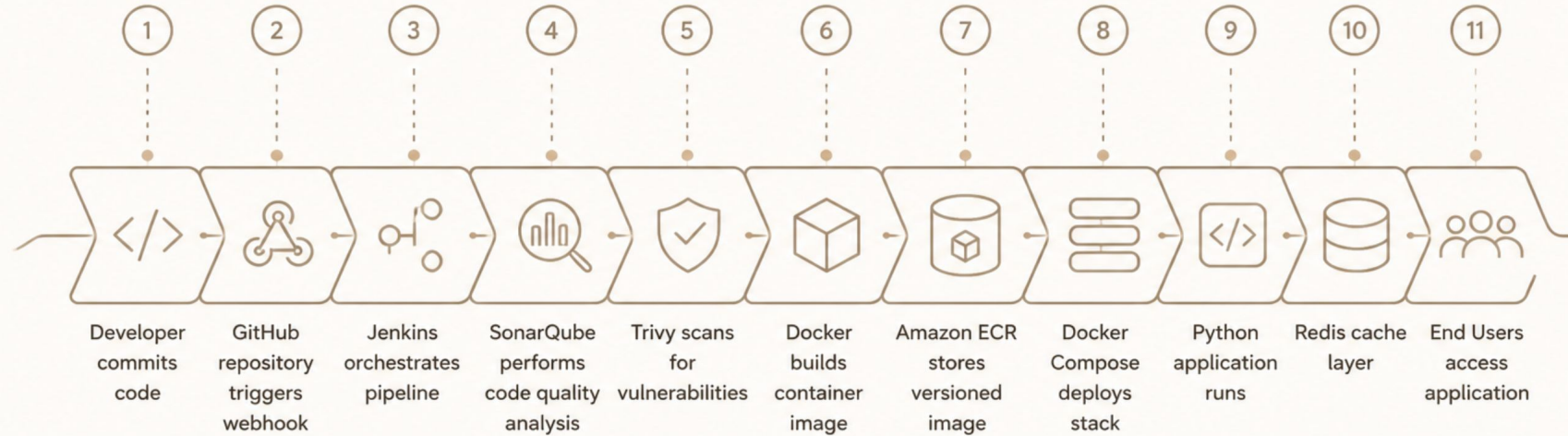
Operations & Safety

- **Secrets Management** – Jenkins credential store + AWS Secrets Manager
- **Rollback Strategy** – Re-deploy previous ECR image tag on failure
- **Artifact Retention** – Keep last 10 builds; archive reports
- **Build History** – Audit trail for compliance and debugging



Enterprise CI/CD End-to-End Flow

ARCHITECTURE OVERVIEW



This enterprise pipeline automates the full software delivery lifecycle — from a developer's commit to a live, validated production deployment — with security, quality, and observability built into every stage.

Common Pipeline Failures

TRUBLESHOOTING GUIDE

Failure	Cause	Symptoms	Resolution
Git Failure	Invalid token or wrong branch	Clone error in console	Verify GitHub credentials in Jenkins store
Build Failure	Dockerfile syntax error	Docker build exits non-zero	Test Dockerfile locally first
Sonar Failure	Quality gate not met	Stage marked failed, report generated	Fix code smells; adjust gate thresholds
Trivy Failure	Critical CVE detected	Vulnerability report with CRITICAL severity	Update base image or patch dependency
ECR Auth Failure	Expired or missing AWS credentials	401 Unauthorized on docker push	Refresh IAM credentials in Jenkins
Deployment Failure	Port conflict or missing env var	Container exits immediately	Check docker logs; validate compose file
Health Check Failure	App not ready or wrong endpoint	HTTP 5xx or timeout	Increase startup delay; fix app config

Project Deliverables

STUDENT SUBMISSION



Source & Config

- GitHub Repository
- Dockerfile
- Docker Compose file
- Jenkinsfile



Security & Registry

- SonarQube Report
- Trivy Scan Report
- Amazon ECR Screenshot



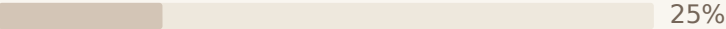
Evidence & Docs

- Pipeline Screenshot
- Running Application
- Technical Documentation
- Demo Video



Evaluation Criteria

GRADING SCORECARD



Pipeline

Jenkinsfile quality, stage design, and automation completeness



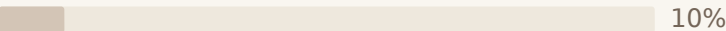
Security

SonarQube quality gate + Trivy vulnerability scan results



Docker

Dockerfile best practices and Compose configuration



Git

Repository structure, commit history, and branching



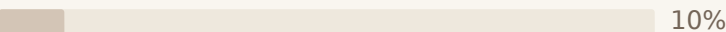
Amazon ECR

Image tagging strategy and lifecycle policy configuration



Deployment

Successful container deployment and health validation



Documentation

Technical docs clarity, demo video, and deliverable completeness

Total: 100%

Each criterion reflects a real-world enterprise DevOps competency. Students are evaluated not just on whether the pipeline runs, but on the quality, security, and professionalism of every component.



A passing score requires demonstrated competency in Security (20%) and Pipeline (25%) — the two highest-weighted areas.

Learning Outcomes

QUESTIONS & DISCUSSION

Upon completing this project, students will demonstrate expertise across the full enterprise DevOps stack:

CI/CD & Jenkins

Design and implement production-grade pipelines as code with parameterization and audit trails.

Docker & Compose

Build immutable container images and orchestrate multi-service stacks with resilience policies.

Security (Sonar + Trivy)

Integrate SAST and vulnerability scanning into automated pipelines for secure software delivery.

Amazon ECR & AWS

Manage private container registries with versioning, lifecycle policies, and IAM-based access control.

Production Automation

Apply DevOps best practices: rollback strategies, secrets management, and deployment observability.

What questions do you have about the pipeline, deliverables, or evaluation criteria?